

Politechnika Świętokrzyska

Wydział Elektrotechniki, Automatyki i Informatyki

Kierunek: Informatyka

Jednostka arytmetyczno-logiczna dla liczb w formacie zmiennoprzecinkowym

Sprawozdanie z projektu

Autorzy: Filip Stępień, Bartłomiej Karkoszka, Rafał Grot

Grupa: 1ID21A

Przedmiot: Projektowanie systemów wbudowanych

Kielce, 2026

Spis treści

1	Cel i zakres projektu	1
2	Środowisko i struktura plików	1
3	Format danych	2
3.1	Reprezentacja liczby	2
3.2	Normalizacja	3
4	Architektura systemu	3
4.1	Interfejs najwyższego poziomu	3
4.2	Przepływ sygnałów	4
5	Opis modułów	5
5.1	Generator impulsów <code>clk_div</code>	5
5.2	Generatory argumentów <code>fp_counter</code>	5
5.3	Opóźnienie startu <code>start_delay</code>	5
5.4	Adaptory <code>fp_counter_bdf</code> i <code>alu_bdf</code>	6
5.5	Jednostka nadrzędna <code>alu</code>	6
5.6	Dodawanie i odejmowanie	6
5.7	Mnożenie	7
6	Przebieg działania	7
7	Kompilacja w Quartusie	8
7.1	Procedura	8
8	Symulacja przebiegów w ModelSimie	8
8.1	Uruchomienie	8
8.2	Interpretacja przebiegów	10
9	Automatyczna weryfikacja obliczeń	11
9.1	Sposób działania testu	11
9.2	Przykładowe przypadki	11
10	Uzyskane rezultaty	12
11	Ograniczenia i możliwy rozwój	13

12 Skrócona instrukcja odtworzenia wyników	13
13 Podsumowanie	13

1 Cel i zakres projektu

Celem projektu było zaprojektowanie oraz przetestowanie w języku VHDL jednostki arytmetyczno-logicznej wykonującej trzy operacje na liczbach zapisanych w autorskim formacie zmiennoprzecinkowym:

- dodawanie,
- odejmowanie,
- mnożenie.

Układ został zbudowany z niezależnych modułów, połączonych na schemacie blokowym `demo.bdf`. Dwa liczniki generują okresowo zmieniające się argumenty wejściowe. Jednostka ALU oblicza wynik wybranej operacji, a sygnał `DONE` informuje o zakończeniu obliczeń. Taka organizacja pozwala obserwować działanie projektu zarówno na przebiegach cyfrowych w ModelSimie, jak i po przypisaniu wyjść do fizycznych wyprowadzeń układu FPGA.

Projekt obejmuje:

- moduły obliczeniowe napisane w VHDL,
- schemat blokowy najwyższego poziomu wykonany w Quartusie,
- skrypt przygotowujący czytelne przebiegi w ModelSimie,
- automatyczny test porównujący wyniki sprzętowe z wartościami referencyjnymi.

2 Środowisko i struktura plików

Projekt przygotowano dla układu **Intel/Altera Cyclone IV E EP4CE115F29C7**. Do syntezy wykorzystano Quartus II, natomiast symulację funkcjonalną przeprowadzono w ModelSim-Altera.

Wszystkie źródła opisujące działanie układu znajdują się w katalogu `src/`. Są to moduły VHDL odpowiedzialne za reprezentację liczb, generowanie argumentów, sterowanie czasem oraz wykonanie operacji arytmetycznych. Dzięki umieszczeniu kodu funkcjonalnego w jednym katalogu można łatwo odróżnić go od plików projektu Quartusa, skryptów testowych i wyników generowanych podczas kompilacji.

Plik `demo.bdf` jest graficznym schematem najwyższego poziomu. Nie zawiera osobnego algorytmu obliczeniowego, lecz określa, jakie moduły VHDL są użyte oraz w jaki sposób połączone są ich porty. Pliki `*.bsf` przechowują symbole bloków widoczne w edytorze schematów Quartusa. Definiują wygląd symbolu i rozmieszczenie jego wejść oraz wyjść, natomiast właściwe działanie bloków pozostaje zapisane w źródłach VHDL.

Tabela 1: Najważniejsze pliki projektu

Plik	Znaczenie
demo.bdf	Graficzny schemat najwyższego poziomu i połączenia modułów.
*.bsf	Symbole modułów VHDL używane w edytorze schematów Quartusa.
fp_alu.qpf, fp_alu.qsf	Konfiguracja projektu Quartus.
src/fp_pkg.vhd	Definicja typu liczby i funkcji pomocniczych.
src/fp_counter.vhd	Generator kolejnych wartości argumentów.
src/clk_div.vhd	Generator impulsów o zadanej liczbie taktów na sekundę.
src/start_delay.vhd	Jednotaktowe opóźnienie rozpoczęcia operacji ALU.
src/fp_add_sub.vhd	Sekwencyjne dodawanie i odejmowanie.
src/fp_mul.vhd	Sekwencyjne mnożenie.
src/fp_normalize.vhd	Normalizacja wyniku.
src/alu.vhd	Wybór operacji i wspólny interfejs ALU.
src/*_bdf.vhd	Adaptory pomiędzy rekordami VHDL a magistralami BDF.
test/wave_scope.do	Konfiguracja i uruchomienie przebiegów.
test/check_counter_results.do	Automatyczna kontrola wyników.

3 Format danych

3.1 Reprezentacja liczby

Argumenty oraz wynik są przechowywane jako rekord `fp_t` złożony z dwóch 16-bitowych liczb ze znakiem:

Listing 1: Format liczby używany w projekcie

```

type fp_t is record
    mantissa : signed(15 downto 0);
    exponent : signed(15 downto 0);
end record;

```

Mantysa jest liczbą stałoprzecinkową w kodzie uzupełnień do dwóch, w formacie Q1.15. Jeden bit reprezentuje znak i część całkowitą, a 15 bitów część ułamkową. Wartość liczby oblicza się ze wzoru:

$$x = \frac{M}{2^{15}} \cdot 2^E, \quad (1)$$

gdzie M jest całkowitą wartością 16-bitowej mantysy, a E jest wykładnikiem.

Tabela 2: Przykładowe wartości formatu

Mantysa hex	M	Wykładnik	Wartość
1000	4096	0	0,125
4000	16384	0	0,5
E000	-8192	0	-0,25
4000	16384	1	1,0

Format jest prostszy od IEEE 754. Nie zawiera wartości specjalnych *NaN* i nieskończoności ani bitu niejawnego. Dzięki temu algorytmy arytmetyczne są czytelne, a przebiegi łatwe do zinterpretowania.

3.2 Normalizacja

Niezerowy wynik jest normalizowany tak, aby wartość bezwzględna mantysy znajdowała się w przedziale:

$$0,5 \leq |\text{mantysa}| < 1. \quad (2)$$

Jeżeli mantysa jest zbyt mała, moduł `fp_normalize` przesuwają ją w lewo i jednocześnie zmniejsza wykładnik. Gdy wynik jest równy zero, zarówno mantysa, jak i wykładnik są zerowane.

4 Architektura systemu

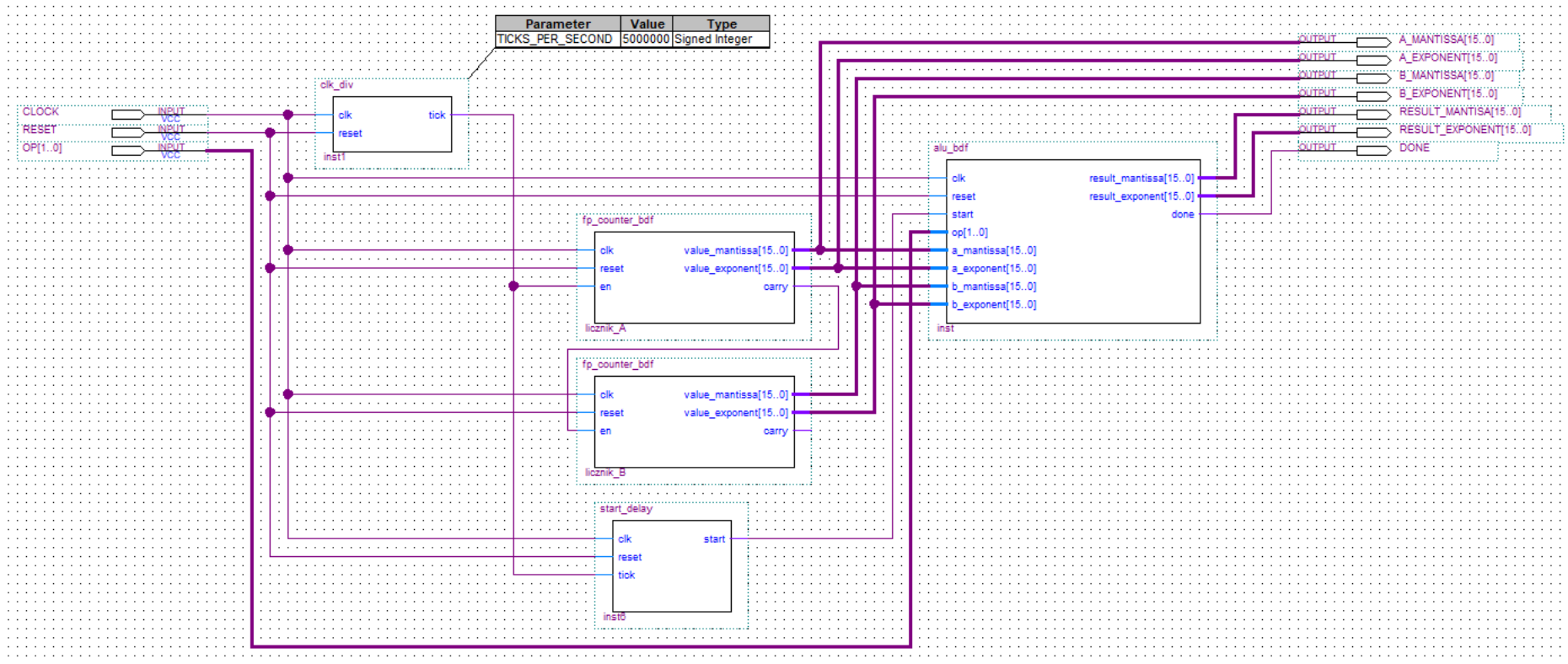
4.1 Interfejs najwyższego poziomu

Tabela 3: Porty schematu `demo.bdf`

Port	Kierunek	Znaczenie
CLOCK	wejście	Zegar systemowy 50 MHz.
RESET	wejście	Asynchroniczny reset aktywny stanem wysokim.
OP[1..0]	wejście	Wybór operacji: 00 – dodawanie, 01 – odejmowanie, 10 – mnożenie.
A_MANTISSA[15..0]	wyjście	Mantysa pierwszego argumentu.
A_EXPONENT[15..0]	wyjście	Wykładnik pierwszego argumentu.
B_MANTISSA[15..0]	wyjście	Mantysa drugiego argumentu.
B_EXPONENT[15..0]	wyjście	Wykładnik drugiego argumentu.
RESULT_MANTISA[15..0]	wyjście	Mantysa wyniku. Nazwa portu jest zgodna ze schematem.
RESULT_EXPONENT[15..0]	wyjście	Wykładnik wyniku.
DONE	wyjście	Jednotaktowy impuls zakończenia operacji.

4.2 Przepływ sygnałów

Generator `clk_div` tworzy impuls `tick`. Impuls zwiększa licznik A. Po wykonaniu pełnego obiegu przez licznik A jego sygnał `carry` zwiększa licznik B. W ten sposób układ automatycznie przechodzi przez wszystkie pary argumentów. Ten sam impuls, opóźniony o jeden takt przez `start_delay`, rozpoczyna obliczenie ALU.



Rysunek 1: Schemat `demo.bdf` w edytorze Block Diagram/Schematic File programu Quartus. Widoczne są moduły projektu, magistrale danych oraz połączenia zegara, resetu, `tick`, `carry` i `start`.

5 Opis modułów

5.1 Generator impulsów `clk_div`

Moduł nie tworzy nowej domeny zegarowej. Zlicza zbocza narastające zegara `CLOCK` i generuje jednotaktowy sygnał zezwolenia `tick`. Liczba taktów pomiędzy impulsami wynosi:

$$N = \frac{\text{CLOCK_HZ}}{\text{TICKS_PER_SECOND}}. \quad (3)$$

Na schemacie przyjęto:

$$\text{CLOCK_HZ} = 50\,000\,000, \quad (4)$$

$$\text{TICKS_PER_SECOND} = 5\,000\,000, \quad (5)$$

$$N = 10. \quad (6)$$

Przy zegarze 50 MHz okres zegara wynosi 20 ns, dlatego nowy impuls `tick` pojawia się co:

$$10 \cdot 20 \text{ ns} = 200 \text{ ns}. \quad (7)$$

Parametr można zmniejszyć podczas pracy z fizycznym oscyloskopem, aby zmiany były wolniejsze. Zwiększona wartość użyta w projekcie znacznie skraca czas symulacji.

5.2 Generatory argumentów `fp_counter`

Oba liczniki mają parametr `WIDTH=4`. Wykorzystują 16 stanów i generują następujący cykl mantysy:

$$0, \frac{1}{8}, \frac{2}{8}, \dots, \frac{7}{8}, -1, -\frac{7}{8}, \dots, -\frac{1}{8}. \quad (8)$$

Wykładnik argumentów jest równy zero. Wartości mantysy zmieniają się zatem od -1 do $0,875$ ze skokiem $0,125$. Po przejściu z ostatniego stanu do zera licznik A wystawia `carry`. Sygnał ten jest podłączony do wejścia `en` licznika B.

Takie połączenie działa jak dwucyfrowy licznik:

- A jest szybką cyfrą i zmienia się przy każdym impulsie `tick`,
- B jest wolną cyfrą i zmienia się po pełnych 16 krokach A,
- w czasie jednego pełnego przebiegu sprawdzanych jest $16 \cdot 16 = 256$ par argumentów.

5.3 Opóźnienie startu `start_delay`

Licznik A aktualizuje argument na zboczu zegara wywołanym przez `tick`. Jednostka ALU nie powinna w tym samym zboczu pobierać starej wartości. Dlatego `start_delay` opóźnia rozpoczęcie operacji o jeden takt zegara:

Takt	k	$k + 1$	$k + 2$	$k + 3$
tick	1	0	0	0
zmiana A/B	tak	stabilne	stabilne	stabilne
start	0	1	0	0

ALU rozpoczyna zatem pracę dopiero wtedy, gdy oba argumenty są już stabilne.

5.4 Adaptery `fp_counter_bdf` i `alu_bdf`

Wewnętrzne moduły VHDL używają rekordów typu `fp_t`. Edytor schematów Quartusa wygodniej obsługuje osobne magistrale `std_logic_vector`. Adaptery rozdzielają rekordy na mantysę i wykładnik oraz składają je ponownie przed przekazaniem do ALU.

Adaptery nie wykonują obliczeń i nie wprowadzają opóźnień. Stanowią jedynie warstwę dopasowującą interfejs kodu VHDL do symboli użytych w pliku BDF. Na ich podstawie Quartus korzysta z symboli `fp_counter_bdf.bsf` i `alu_bdf.bsf`, które można wstawić na schemat oraz połączyć zwykłymi przewodami i magistralami. Pozostałe symbole, między innymi `clk_div.bsf` i `start_delay.bsf`, odpowiadają bezpośrednio portom modułów VHDL.

Rozdzielenie to pozwala zachować czytelny kod źródłowy z rekordem `fp_t`, a jednocześnie przedstawiać najważniejsze połączenia całego systemu w graficznej postaci na schemacie `demo.bdf`.

5.5 Jednostka nadrzędna `alu`

Moduł `alu` uruchamia równolegle:

- blok dodawania/odejmowania `fp_add_sub`,
- blok mnożenia `fp_mul`.

Sygnał `OP` wybiera wynik i sygnał `done` właściwego bloku. Kodowanie operacji przedstawia tabela 4.

Tabela 4: Kodowanie wejścia `OP`

OP	Operacja
00	$A + B$
01	$A - B$
10	$A \cdot B$
11	kod niewykorzystywany

5.6 Dodawanie i odejmowanie

Blok `fp_add_sub` jest maszyną stanów. Operacja przebiega w czterech etapach:

Tabela 5: Stany bloku dodawania i odejmowania

Stan	Działanie
S_IDLE	Oczekiwanie na start i zapamiętanie argumentów.
S_ALIGN	Wyrównanie wykładników przez przesunięcie mantysy liczby o mniejszym wykładniku.
S_COMPUTE	Dodanie lub odjęcie rozszerzonych mantys oraz obsługa przepełnienia.
S_NORMALIZE	Normalizacja, zapis wyniku i wygenerowanie done .

Przed wykonaniem działania wykładniki muszą być równe. Mantysa liczby o mniejszym wykładniku jest przesuwana arytmetycznie w prawo, a jej wykładnik zwiększany. Po wyrównaniu wykonywane jest dodawanie lub odejmowanie na 17 bitach. Dodatkowy bit umożliwia wykrycie wyniku wykraczającego poza zakres Q1.15. W takim przypadku mantysa jest przesuwana w prawo, a wykładnik zwiększany o jeden.

5.7 Mnożenie

Blok `fp_mul` działa w trzech stanach:

1. zapamiętuje argumenty,
2. mnoży dwie 16-bitowe mantysy i dodaje wykładniki,
3. normalizuje wynik i zgłasza zakończenie.

Iloczyn dwóch liczb Q1.15 ma 32 bity i skalę Q2.30. Aby wrócić do formatu Q1.15, wybierane są bity 30 `downto` 15. Wartość przed normalizacją można zapisać jako:

$$M_R = \frac{M_A \cdot M_B}{2^{15}}, \quad E_R = E_A + E_B. \quad (9)$$

Po tej operacji wynik jest przekazywany do wspólnego modułu normalizującego.

6 Przebieg działania

Po zwolnieniu resetu system pracuje autonomicznie:

1. `clk_div` odmierza dziesięć taktów zegara,
2. impuls `tick` zwiększa argument A,
3. po pełnym obiegu A zwiększany jest argument B,
4. takt później `start_delay` generuje `start`,
5. ALU zapamiętuje A, B oraz wykonuje operację wybraną przez OP,
6. po zakończeniu wystawia wynik oraz jednotaktowy impuls `DONE`.

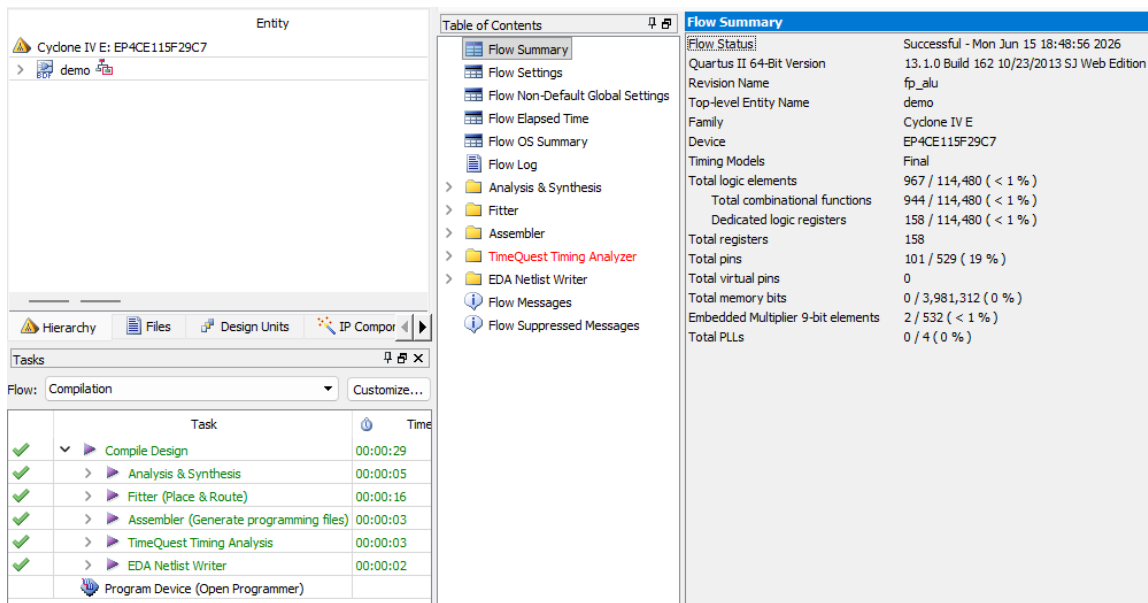
Okres 200 ns pomiędzy kolejnymi impulsami `tick` jest większy od latencji każdego bloku arytmetycznego, dlatego bieżąca operacja kończy się przed rozpoczęciem następnej.

7 Kompilacja w Quartusie

7.1 Procedura

1. Otworzyć plik projektu `fp_alu.qpf`.
2. Sprawdzić, czy jednostką najwyższego poziomu jest `demo`.
3. Wybrać `Processing -> Start Compilation`.
4. Po zakończeniu sprawdzić liczbę błędów w oknie komunikatów.
5. Do symulacji funkcjonalnej wygenerować plik `simulation/qsim/fp_alu.vo`.

Pełna kompilacja sprawdzonej wersji projektu kończy się bez błędów.



Rysunek 2: Wynik kompilacji projektu w Quartusie. Kompilacja jednostki najwyższego poziomu `demo` zakończyła się bez błędów.

8 Symulacja przebiegów w ModelSimie

8.1 Uruchomienie

Skrypt należy uruchomić z katalogu `simulation/qsim`:

Listing 2: Uruchomienie symulacji przebiegów

```
do ../../test/wave_scope.do
```

Skrypt automatycznie:

- kompiluje netlistę `fp_alu.vo`,
- ładuje projekt `work.demo`,
- generuje zegar 50 MHz,
- wymusza reset i kolejne wartości `OP`,
- dodaje najważniejsze sygnały do okna Wave,
- ustawia mantysy jako analogowe przebiegi schodkowe,

- uruchamia symulację na $50\ \mu\text{s}$.

Tabela 6: Wymuszenia użyte przez `wave_scope.do`

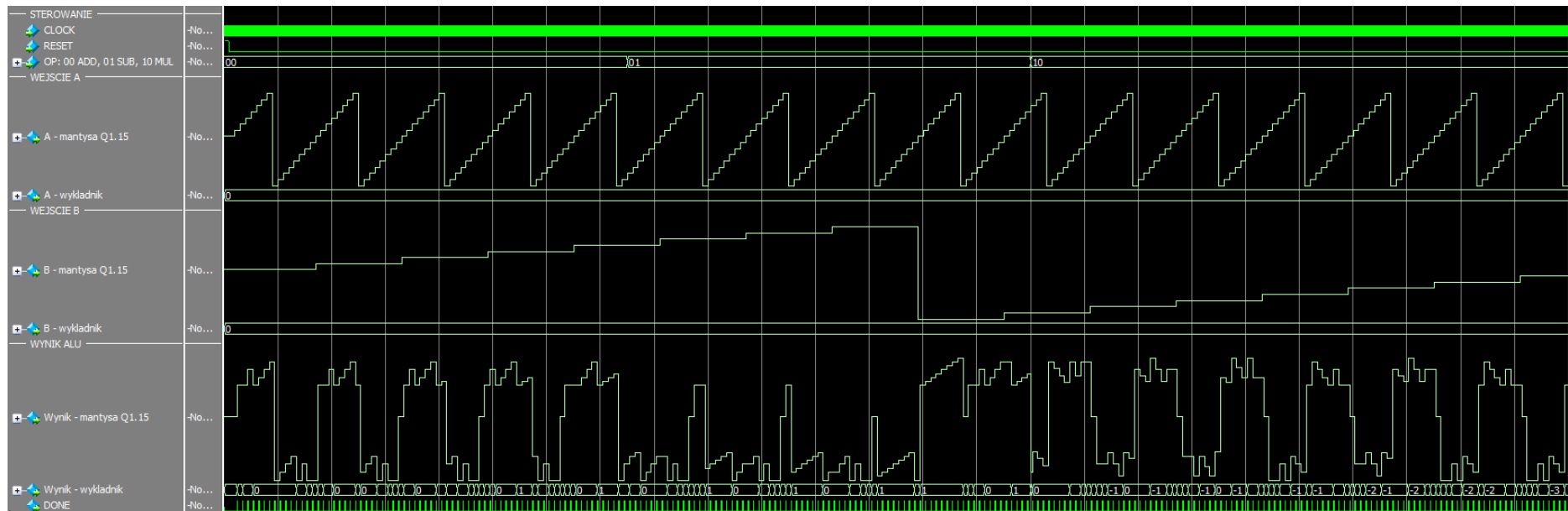
Czas	Sygnal	Wartość
0–200 ns	RESET	1
od 200 ns	RESET	0
0–15 μs	OP	00 – dodawanie
15–30 μs	OP	01 – odejmowanie
30–50 μs	OP	10 – mnożenie

8.2 Interpretacja przebiegów

Na wykresie A tworzy szybką piłę schodkową, ponieważ jego licznik jest zwiększany przy każdym impulsie `tick`. B zmienia się 16 razy wolniej, gdyż jest zwiększane dopiero po pełnym cyklu A. Wynik zależy od operacji wskazanej przez `OP`:

- dla `OP=00` wynik jest sumą A i B; ponieważ B pozostaje chwilowo stałe, przebieg przypomina A przesunięte o wartość B,
- dla `OP=01` wynik jest różnicą A i B; odejmowanie stałego fragmentu B przesuwa przebieg A w dół,
- dla `OP=10` B skaluje przebieg A; dodatnie B zachowuje jego kierunek, ujemne odwraca znak, a wartość bezwzględna B określa amplitudę.

Mantysę i wykładnik należy analizować łącznie. Sama mantysa może znaleźć po normalizacji, mimo że rzeczywista wartość wyniku wzrosła. Przykładowo liczba 1 jest reprezentowana jako mantysa 0,5 i wykładnik 1. Z tego powodu widoczny kształt mantysy wyniku może zawierać dodatkowe uskoki w chwilach zmiany wykładnika, mimo że rzeczywista wartość operacji zmienia się zgodnie z opisanymi zależnościami. Schodkowy wygląd wyniku natomiast z tego, że ModelSim rysuje kolejne cyfrowe wartości magistrali jako przebieg *Analog (Step)*.



Rysunek 3: Okno Wave po wykonaniu `wave_scope.do`. Mantysy A, B i wyniku przedstawiono jako przebiegi *Analog (Step)*; widoczne są również wykładniki, DONE, OP, RESET i CLOCK.

9 Automatyczna weryfikacja obliczeń

9.1 Sposób działania testu

Samo obejrzenie przebiegów pozwala ocenić ogólny kształt sygnałów, ale nie gwarantuje poprawności każdej wartości. Dlatego przygotowano skrypt `check_counter_results.do`.

Uruchomienie:

Listing 3: Uruchomienie automatycznego testu

```
do ../../test/check_counter_results.do
```

Po każdym impulsie DONE skrypt:

1. odczytuje mantysy i wykładniki A, B oraz wyniku,
2. zamienia zapis sprzętowy na wartości rzeczywiste,
3. oblicza oczekiwany wynik dla aktualnego OP,
4. porównuje wynik sprzętowy z referencyjnym,
5. zapisuje wiersz **PASS** albo **FAIL**.

Wartość odczytana z portów jest rekonstruowana zgodnie ze wzorem:

$$x_{\text{ModelSim}} = \frac{\text{signed}(\text{mantissa})}{32768} \cdot 2^{\text{signed}(\text{exponent})}. \quad (10)$$

Porównanie wykorzystuje tolerancję 10^{-9} . Dla wartości występujących w tym teście nie jest potrzebne dodatkowe zaokrąglanie, ponieważ są one wielokrotnościami potęg liczby 2 i mają dokładną reprezentację binarną.

9.2 Przykładowe przypadki

Tabela 7: Przykładowe wyniki sprawdzane przez skrypt

Operacja	A	B	Oczekiwano	Otrzymano
ADD	0,875	0,125	1,000	1,000
SUB	-0,750	0,500	-1,250	-1,250
MUL	0,625	-0,875	-0,546875	-0,546875

Pełny dziennik jest zapisywany do pliku:

```
build/counter_results.log
```

W zweryfikowanej wersji projektu otrzymano:

Listing 4: Podsumowanie automatycznego testu

```
SUMMARY: checked=248 passed=248 failed=0  
COUNTER/ALU CHECK PASSED
```

Liczba 248 odpowiada operacjom zakończonym w czasie 50 μ s przy zastosowanych wymuszeniach. Nie jest to ograniczenie liczby kombinacji wejściowych, lecz rezultat wybranego czasu symulacji.

```
# 220 MUL A=-0.50000 B=-0.37500 expected= 0.18750 ALU= 0.18750 m= 24576 e= -2 PASS
# 221 MUL A=-0.37500 B=-0.37500 expected= 0.14063 ALU= 0.14063 m= 18432 e= -2 PASS
# 222 MUL A=-0.25000 B=-0.37500 expected= 0.09375 ALU= 0.09375 m= 24576 e= -3 PASS
# 223 MUL A=-0.12500 B=-0.37500 expected= 0.04688 ALU= 0.04688 m= 24576 e= -4 PASS
# 224 MUL A= 0.00000 B=-0.25000 expected= -0.00000 ALU= 0.00000 m= 0 e= 0 PASS
# 225 MUL A= 0.12500 B=-0.25000 expected= -0.03125 ALU= -0.03125 m=-32768 e= -5 PASS
# 226 MUL A= 0.25000 B=-0.25000 expected= -0.06250 ALU= -0.06250 m=-32768 e= -4 PASS
# 227 MUL A= 0.37500 B=-0.25000 expected= -0.09375 ALU= -0.09375 m=-24576 e= -3 PASS
# 228 MUL A= 0.50000 B=-0.25000 expected= -0.12500 ALU= -0.12500 m=-32768 e= -3 PASS
# 229 MUL A= 0.62500 B=-0.25000 expected= -0.15625 ALU= -0.15625 m=-20480 e= -2 PASS
# 230 MUL A= 0.75000 B=-0.25000 expected= -0.18750 ALU= -0.18750 m=-24576 e= -2 PASS
# 231 MUL A= 0.87500 B=-0.25000 expected= -0.21875 ALU= -0.21875 m=-28672 e= -2 PASS
# 232 MUL A=-1.00000 B=-0.25000 expected= 0.25000 ALU= 0.25000 m= 16384 e= -1 PASS
# 233 MUL A=-0.87500 B=-0.25000 expected= 0.21875 ALU= 0.21875 m= 28672 e= -2 PASS
# 234 MUL A=-0.75000 B=-0.25000 expected= 0.18750 ALU= 0.18750 m= 24576 e= -2 PASS
# 235 MUL A=-0.62500 B=-0.25000 expected= 0.15625 ALU= 0.15625 m= 20480 e= -2 PASS
# 236 MUL A=-0.50000 B=-0.25000 expected= 0.12500 ALU= 0.12500 m= 16384 e= -2 PASS
# 237 MUL A=-0.37500 B=-0.25000 expected= 0.09375 ALU= 0.09375 m= 24576 e= -3 PASS
# 238 MUL A=-0.25000 B=-0.25000 expected= 0.06250 ALU= 0.06250 m= 16384 e= -3 PASS
# 239 MUL A=-0.12500 B=-0.25000 expected= 0.03125 ALU= 0.03125 m= 16384 e= -4 PASS
# 240 MUL A= 0.00000 B=-0.12500 expected= -0.00000 ALU= 0.00000 m= 0 e= 0 PASS
# 241 MUL A= 0.12500 B=-0.12500 expected= -0.01563 ALU= -0.01563 m=-32768 e= -6 PASS
# 242 MUL A= 0.25000 B=-0.12500 expected= -0.03125 ALU= -0.03125 m=-32768 e= -5 PASS
# 243 MUL A= 0.37500 B=-0.12500 expected= -0.04688 ALU= -0.04688 m=-24576 e= -4 PASS
# 244 MUL A= 0.50000 B=-0.12500 expected= -0.06250 ALU= -0.06250 m=-32768 e= -4 PASS
# 245 MUL A= 0.62500 B=-0.12500 expected= -0.07813 ALU= -0.07813 m=-20480 e= -3 PASS
# 246 MUL A= 0.75000 B=-0.12500 expected= -0.09375 ALU= -0.09375 m=-24576 e= -3 PASS
# 247 MUL A= 0.87500 B=-0.12500 expected= -0.10938 ALU= -0.10938 m=-28672 e= -3 PASS
# 248 MUL A=-1.00000 B=-0.12500 expected= 0.12500 ALU= 0.12500 m= 16384 e= -2 PASS
# SUMMARY: checked=248 passed=248 failed=0
# SUMMARY: checked=248 passed=248 failed=0
# COUNTER/ALU CHECK PASSED
```

VSIM 4>

Rysunek 4: Okno Transcript po wykonaniu `check_counter_results.do`. Końcowe podsumowanie zawiera `checked=248 passed=248 failed=0` oraz komunikat `COUNTER/ALU CHECK PASSED`.

10 Uzyskane rezultaty

Przeprowadzona kompilacja i symulacja potwierdziły:

- poprawne generowanie argumentów A i B,
- właściwe opóźnienie sygnału startu względem zmiany argumentów,
- poprawne działanie dodawania, odejmowania i mnożenia,
- prawidłową normalizację mantysy i aktualizację wykładnika,
- generowanie impulsu DONE po zakończeniu obliczenia,
- zgodność 248 kolejnych wyników z modelem referencyjnym.

Przebiegi analogowe w ModelSimie ułatwiają ocenę działania systemu: argument A jest szybką piłą, B wolną piłą, a wynik tworzy kształt zależny od operacji. Automatyczny test uzupełnia tę ocenę o dokładne sprawdzenie wartości liczbowych.

11 Ograniczenia i możliwy rozwój

Aktualna wersja jest demonstratorem jednostki ALU, dlatego ma kilka świadomie przyjętych ograniczeń:

- format nie jest zgodny z IEEE 754,
- nie występują wartości specjalne ani obsługa wyjątków,
- wykładniki argumentów z liczników są stale równe zero,
- nie zastosowano bitów ochronnych i rozbudowanego zaokrąglania,
- kod OP=11 nie realizuje operacji,
- przed implementacją na płycie należy przypisać piny i dodać ograniczenia czasowe.

Projekt można rozwinąć przez dodanie dzielenia, testów dla różnych wykładników, obsługi przepełnienia wykładnika, trybów zaokrąglania oraz interfejsu umożliwiającego podawanie argumentów z zewnątrz.

12 Skrócona instrukcja odtworzenia wyników

1. Otworzyć `fp_alu.qpf` w Quartusie.
2. Uruchomić pełną kompilację projektu.
3. Upewnić się, że w `simulation/qsim` znajduje się `fp_alu.vo`.
4. Uruchomić ModelSim-Altera w katalogu `simulation/qsim`.
5. Wpisać do `../test/wave_scope.do`, aby obejrzeć przebiegi.
6. Wpisać do `../test/check_counter_results.do`, aby wykonać kontrolę liczbową.
7. Sprawdzić podsumowanie w oknie Transcript oraz dziennik `build/counter_results.log`.

13 Podsumowanie

W ramach projektu wykonano modułową jednostkę ALU operującą na liczbach z 16-bitową mantysą Q1.15 i 16-bitowym wykładnikiem. Układ realizuje dodawanie, odejmowanie oraz mnożenie, a generatory argumentów pozwalają na ciągłą obserwację jego działania bez zewnętrznego źródła danych.

Połączenie schematu blokowego Quartusa, czytelnych przebiegów ModelSim oraz automatycznej kontroli wartości zapewnia trzy poziomy weryfikacji: strukturalny, wizualny i liczbowy. Wszystkie sprawdzone przypadki zakończyły się wynikiem poprawnym.